RICE UNIVERSITY

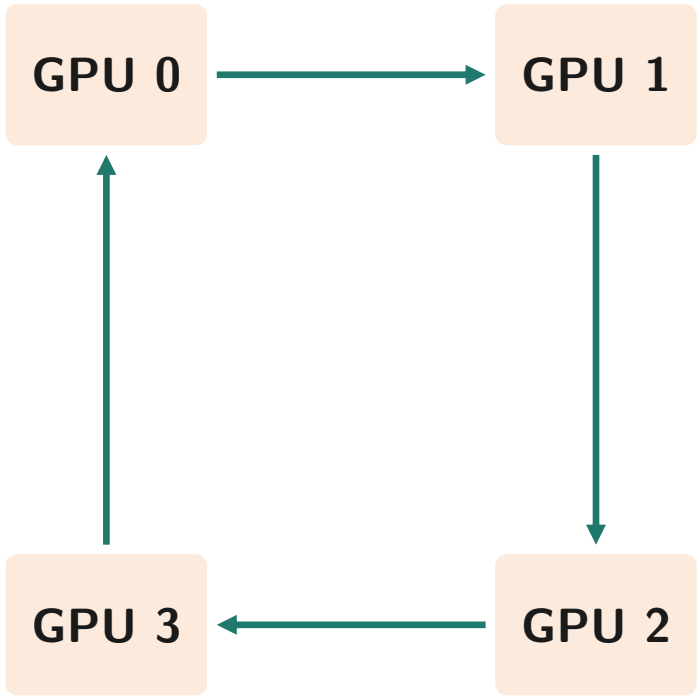
NCCL and GPU Communication

How GPUs add up their gradients, how fast that can be, and how to test it

Yuke Wang

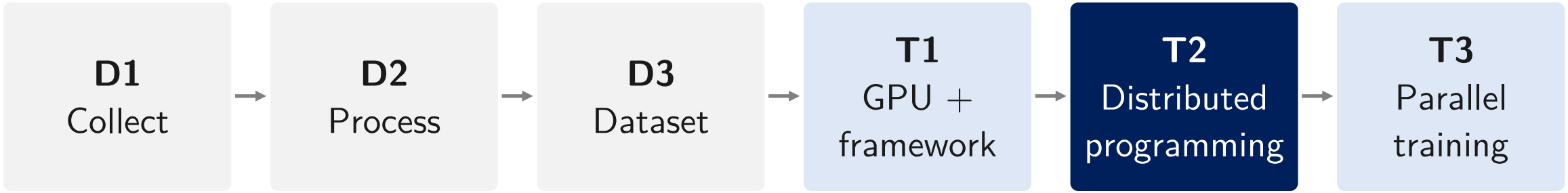
Rice University

COMP 329 · Spring 2027 · Week 11, Lecture 1



Ring all-reduce: each GPU talks only to its neighbour

Where we are in the pipeline



In our pipeline

Today: collectives **between GPUs** with NCCL. T2 checkpoint this week: correct collectives on small tensors.

Last time: processes, ranks, collective semantics, the α–β model, mpi4py. Next time: many servers.

Learning goals for today

- Explain what **NCCL** does and set up **communicators** and **process groups** in torch.distributed.
- Derive **ring all-reduce** step by step, and its cost in the α–β model.
- Explain why the ring is **bandwidth-optimal**, and when a **tree** wins instead.
- Relate all-reduce to **reduce-scatter + all-gather**, and **bus** to **algorithm** bandwidth.
- Test a collective for **correct results**, **completion**, and **consistent ordering**.

A puzzle to start

Each of p GPUs holds 1 GB of gradients.
Goal: every GPU gets the elementwise sum.
Every GPU has the same link speed to its neighbours.

Going from $p = 2$ to $p = 64$ GPUs, roughly how much longer does the sum take?

Key idea

Less than 2× longer. With a ring, time $\propto 2(p-1)/p$: 1.0 at $p = 2$, 1.75 at $p = 8$, 1.97 at $p = 64$ — for large messages.

Recap: collectives and the α–β model

Collective	Each rank starts with	Each rank ends with
Broadcast	root: n bytes	the root's n bytes
All-reduce	own n bytes	sum of all p vectors
Reduce-scatter	own n bytes	one n/p slice of the sum
All-gather	one n/p slice	all p slices (n bytes)

$$T(n) = \alpha + n \,/\, \beta$$

α: latency per message

β: bandwidth per link

Key idea

Today: GPUs instead of CPUs, but the same model — count **steps** (α) and **bytes per link** (β).

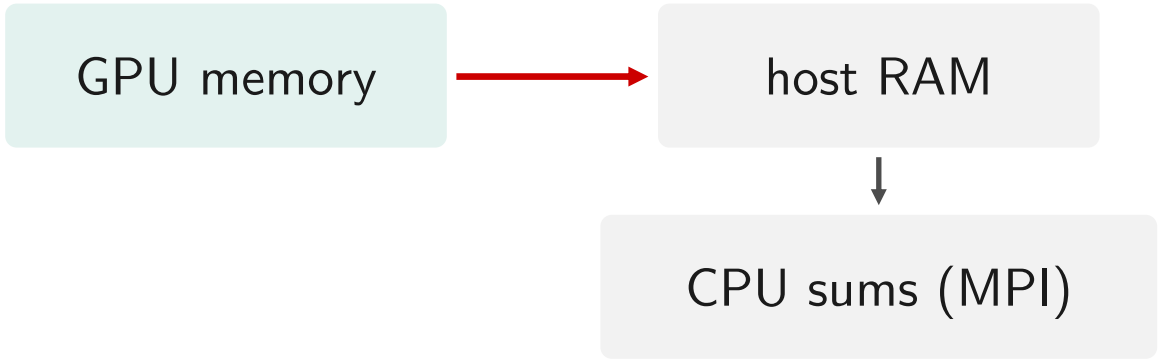
Part 1

NCCL and communicators

Who talks to whom, and over which wires?

Why a GPU collective library?

Old path: through the CPU



PCIe $\approx$ 64 GB/s, crossed twice

NCCL: GPU to GPU



NVLink 450 GB/s per direction; the sum is a GPU kernel

Key idea

NCCL collectives are **GPU kernels**: data never leaves device memory unless it must cross the network.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). PCI-SIG PCIe 5.0.

What NCCL provides

Your program: PyTorch, JAX, … (one process per GPU)

NCCL

collectives · send/recv · topology detection · rings and trees

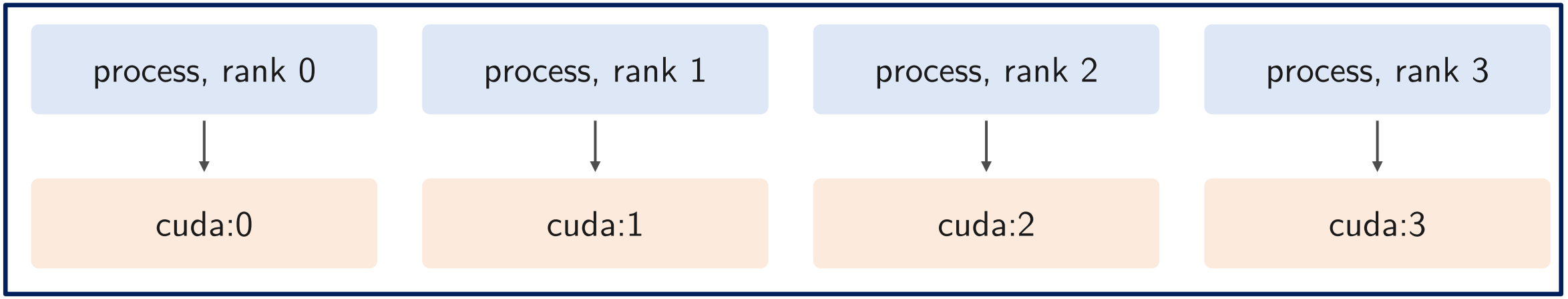
NVLink / NVSwitch	**PCIe peer-to-peer**	**InfiniBand / RoCE**	**TCP sockets**
inside a server	inside a server	between servers	fallback

Watch out

Every NCCL call **enqueues work on a CUDA stream** and returns immediately.

Source: NVIDIA NCCL User Guide: Collective Operations; Communicators; CUDA Stream Semantics.

Communicators: ranks bound to devices



one communicator: 4 ranks, 4 devices

- Created **collectively**: a shared unique ID, then every rank joins with (ID, rank, size).
- Rule: **one rank per GPU**; two ranks on one device may hang.

Source: NVIDIA NCCL User Guide, "Creating a Communicator".

torch.distributed with the NCCL backend

hello_nccl.py

```
import os, torch
import torch.distributed as dist

local = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local)
dist.init_process_group(backend="nccl")
rank = dist.get_rank()
p = dist.get_world_size()

x = torch.full((4,), rank * 1.0, device="cuda")
dist.all_reduce(x)        # in place, op=SUM
print(rank, x.tolist())
dist.destroy_process_group()
```

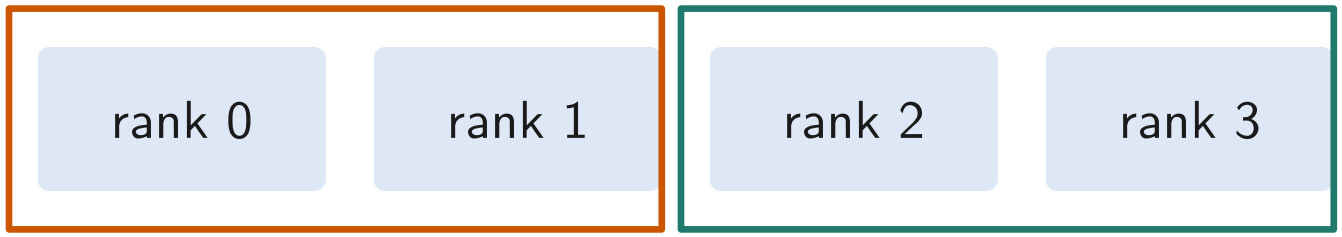
- The launcher sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE`.
- Pick the GPU **before** init.
- $p = 4$: every rank prints [6, 6, 6, 6].

LAUNCH

```
torchrun --nproc-per-node=4
  hello_nccl.py
```

Source: PyTorch documentation, torch.distributed (v2.9).

Process groups: subsets of ranks




```
a = dist.new_group([0, 1])  # ALL ranks call
b = dist.new_group([2, 3])  # same order
mine = a if rank < 2 else b
dist.all_reduce(x, group=mine)
```

RULE

Every rank calls `new_group`, even non-members, and in the **same order** everywhere.

Source: PyTorch documentation, torch.distributed.new_group (v2.9).

The collectives, with sizes

Operation	torch.distributed call	Bytes into each rank
All-reduce	`all_reduce(x)`	$2(p-1)/p \cdot n$
Reduce-scatter	`reduce_scatter_tensor(out, x)`	$(p-1)/p \cdot n$
All-gather	`all_gather_into_tensor(out, x)`	$(p-1)/p \cdot n$
Broadcast	`broadcast(x, src=0)`	n

Key idea

n = the full vector size. Every collective takes `group=` and `async_op=` arguments.

Source: PyTorch documentation, torch.distributed (v2.9). Byte counts: Thakur et al., IJHPCA 2005.

Check yourself

Two minutes with a neighbour.

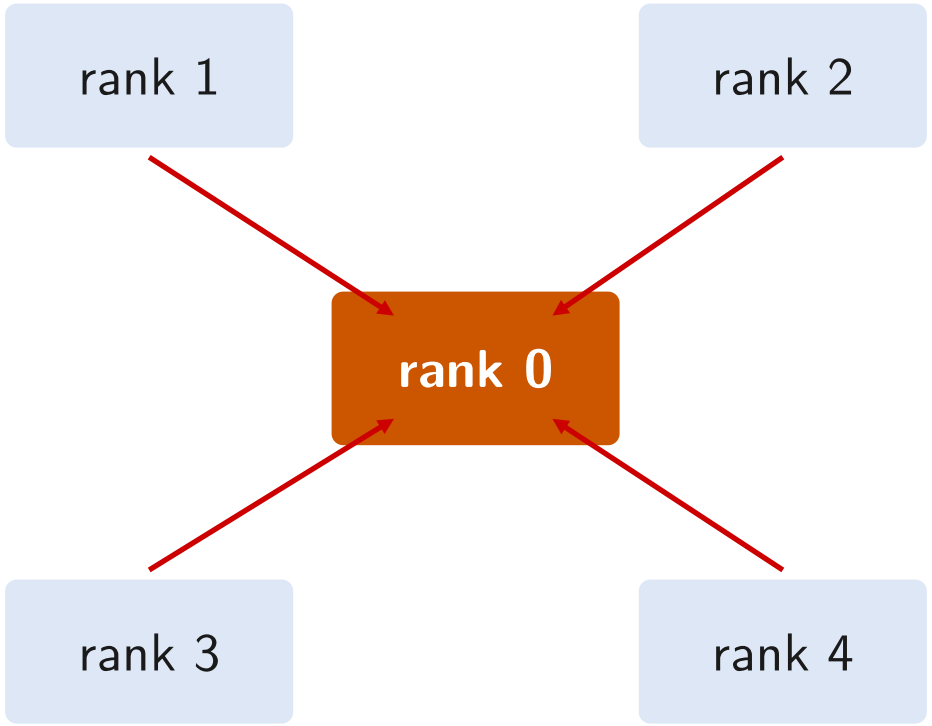
- What is a communicator, and how is one created?
- Ranks 0–2 call `dist.new_group([0, 1, 2])`; rank 3 skips it. What happens?
- You forget `torch.cuda.set_device(local)`. What goes wrong with 4 GPUs?

Part 2

Ring and tree all-reduce

How do p GPUs sum n bytes as fast as possible?

A first idea: gather to one GPU



$$\text{rank 0 receives } (p-1)\cdot n \text{ bytes}$$

$$\text{then sends } (p-1)\cdot n \text{ bytes}$$

$$T \approx 2(p-1)\cdot n \;/\; \beta$$

Watch out

p = 8, n = 1 GB, β = 450 GB/s: T $\approx$ **31 ms**, growing **linearly** in p. One GPU's links do all the work.

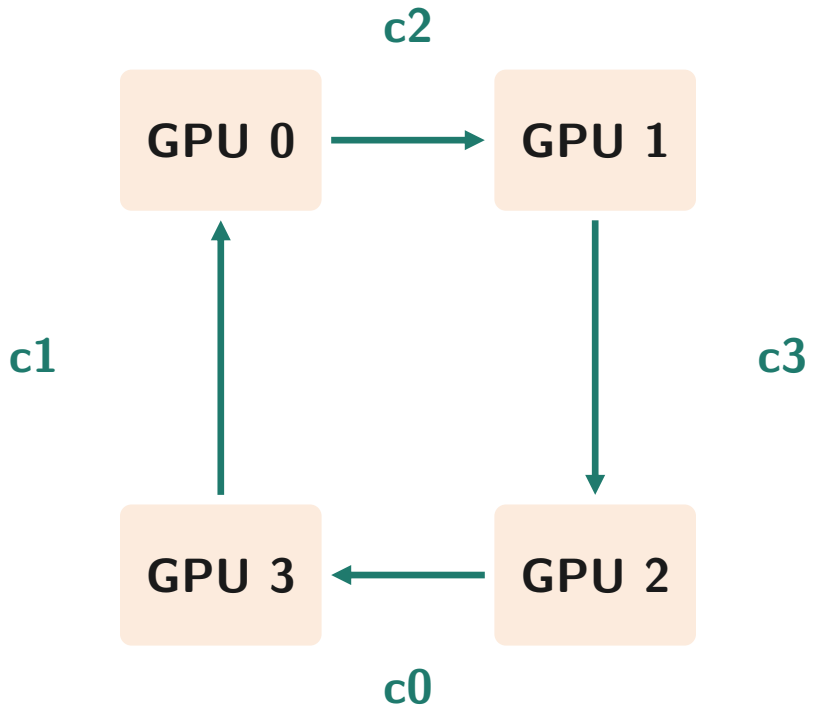
Source: Bandwidth: NVLink 4, 900 GB/s bidirectional per GPU = 450 GB/s per direction (H100 datasheet).

The ring idea: split, then pass along

- Split each GPU's n bytes into **p chunks** of n/p bytes.
- Arrange the GPUs in a **ring**: g sends to $g + 1$, receives from $g - 1$.
- Every step: **every** GPU sends one chunk and receives one — all links busy.
- Phase 1, **reduce-scatter** ($p - 1$ steps): each GPU finishes one summed chunk.
- Phase 2, **all-gather** ($p - 1$ steps): the finished chunks travel around the ring.

Source: Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 69(2), 2009.

Ring all-reduce, phase 1: reduce-scatter

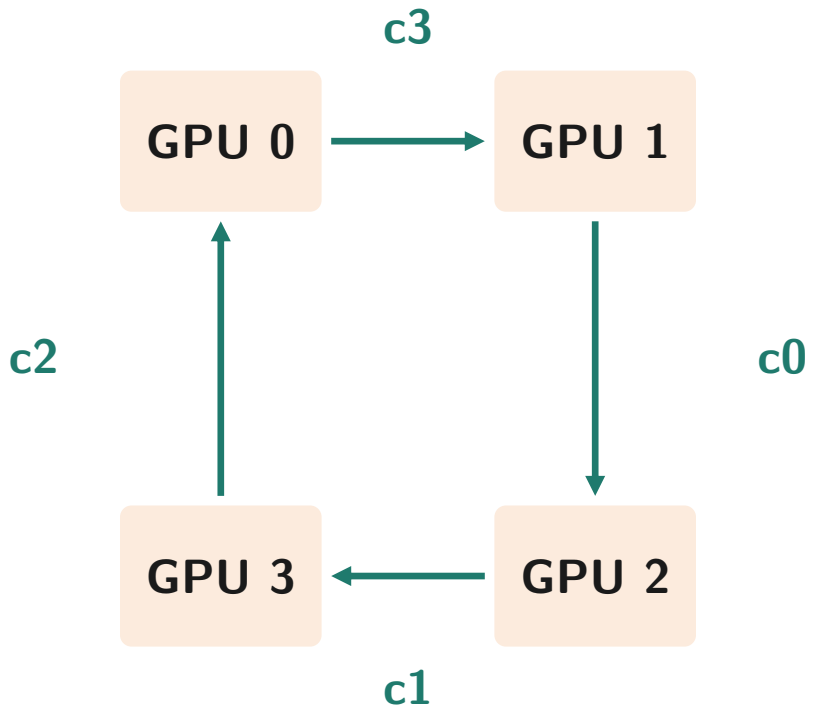


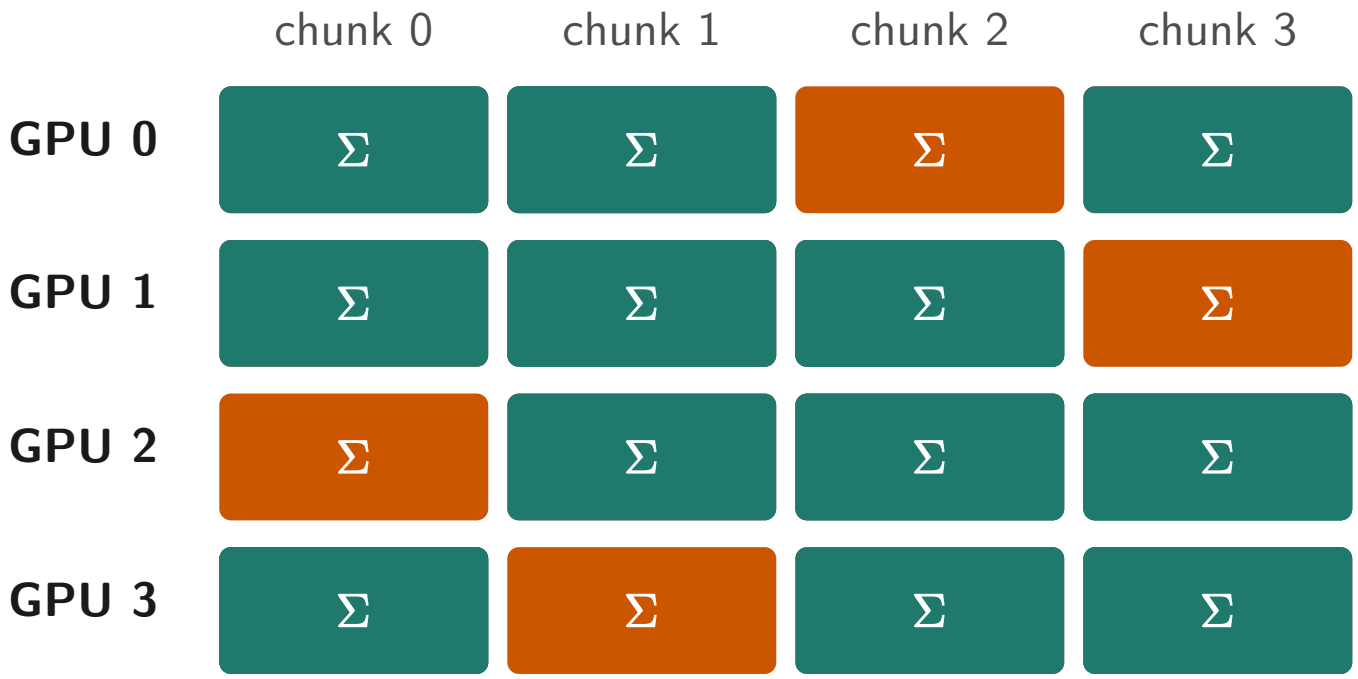
	chunk 0	chunk 1	chunk 2	chunk 3
GPU 0	0	Σ	2+3+0	3+0
GPU 1	0+1	1	Σ	3+0+1
GPU 2	0+1+2	1+2	2	Σ
GPU 3	Σ	1+2+3	2+3	3

Step 3: every GPU g sends chunk g − 2 to GPU g + 1, which adds it to its own copy

orange: just received · green: finished (Σ = all 4 GPUs)

Ring all-reduce, phase 2: all-gather





Step 3: every GPU g sends chunk g − 1 to GPU g + 1, which overwrites its copy

orange: just received · green: finished (Σ = all 4 GPUs)

The cost of ring all-reduce

$$\text{steps:}\quad (p-1)\ \text{reduce-scatter} + (p-1)\ \text{all-gather}$$

$$\text{one step:}\quad \alpha + (n\ /\ p)\ /\ \beta$$

$$T = 2(p-1)\ \alpha\ +\ 2(p-1)/p \cdot n\ /\ \beta$$

latency term
$2(p-1)\alpha$ — grows with p

bandwidth term
factor $2(p-1)/p < 2$ for every p

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Why the ring is bandwidth-optimal

- **Send out** $\geq (p-1)/p \cdot n$: its other chunks are summed elsewhere.
- **Receive** $\geq (p-1)/p \cdot n$: it finished at most one chunk itself.
- Total $\geq 2(p-1)/p \cdot n$ bytes per GPU link (no sums inside the network).

$$T \geq 2(p-1)/p \cdot n \;/\; \beta \quad — \quad \text{the ring meets it exactly}$$

Key idea

No algorithm on these links can beat the ring's **bandwidth term**; only its **latency term**.

Source: Patarasuk & Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," JPDC 69(2), 2009.

Nearly independent of p

p	$2(p-1)/p$	bandwidth term	latency term
2	1.00	2.2 ms	10 µs
8	1.75	3.9 ms	70 µs
64	1.97	4.4 ms	630 µs

$n = 1$ GB

$\beta = 450$ GB/s

$\alpha = 5$ µs (assumed)

Key idea

Large messages: time **almost flat** in p. Small messages: the $2(p-1)\alpha$ term **grows linearly**.

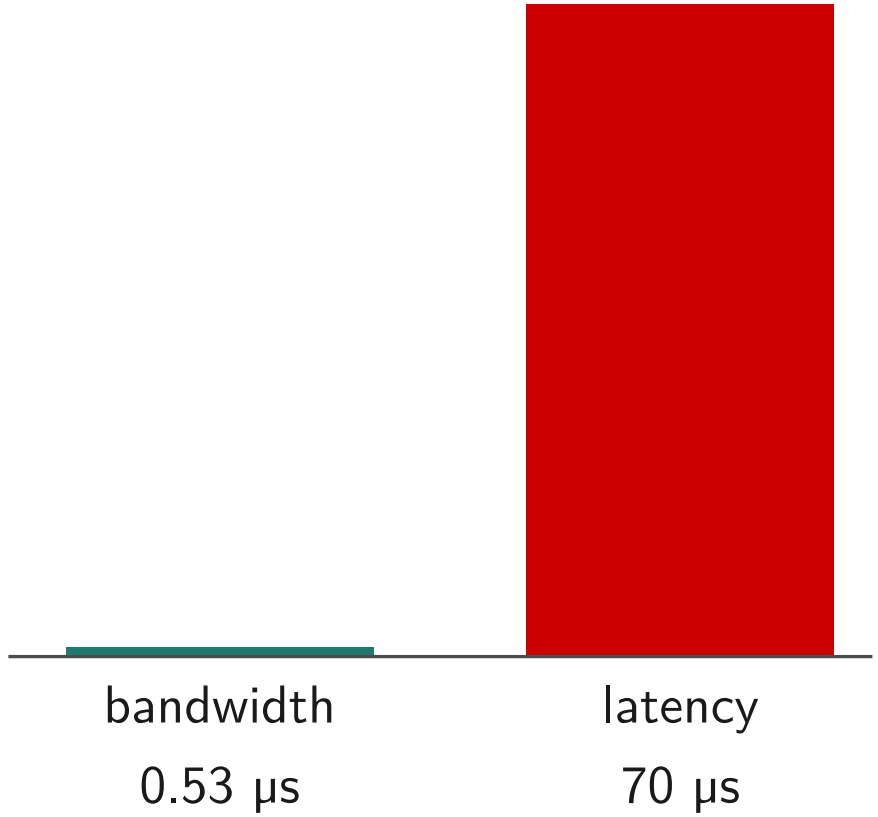
Source: NVLink: H100 datasheet (900 GB/s bidirectional $\Rightarrow$ 450 GB/s per direction); α is an assumed value.

Worked example: our MLP's gradients

- $n = 34{,}338 \times 4 = \mathbf{137{,}352}$ **bytes**
- bandwidth: $2 \cdot 7/8 \cdot n \,/\, 450$ GB/s $\approx \mathbf{0.53}$ **µs**
- latency: $2 \cdot 7 \cdot 5$ µs $= \mathbf{70}$ **µs**

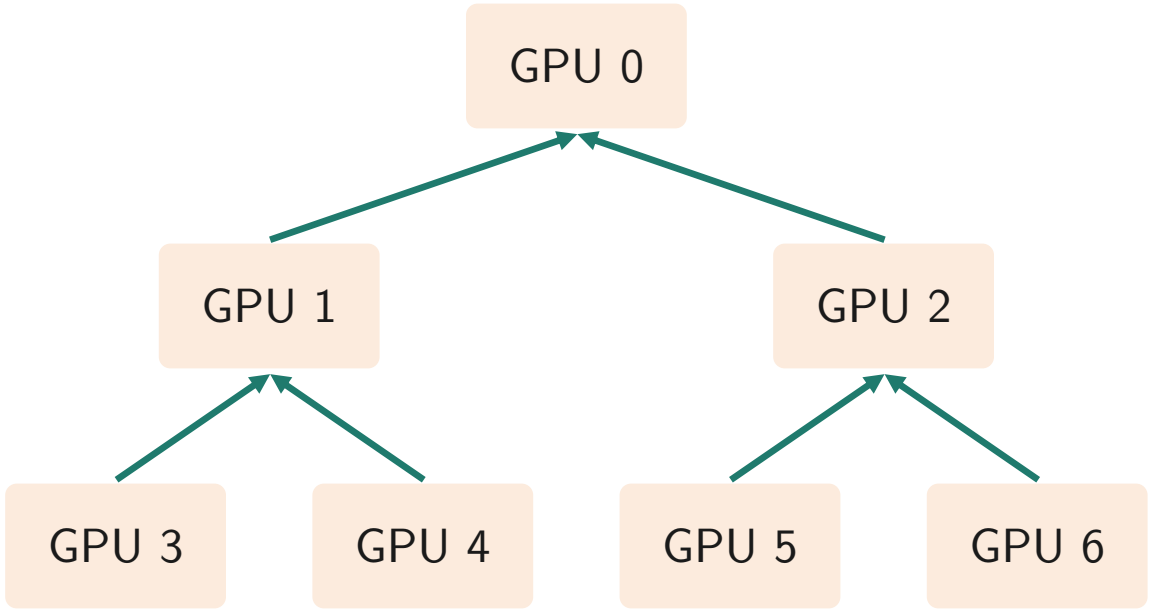
Watch out

Small messages are **latency-bound**: count steps, not bytes.



Source: $p = 8$ GPUs, $\beta = 450$ GB/s (H100 datasheet), $\alpha = 5$ µs per step (assumed).

Trees: fewer steps for small messages



step 1: partial sums move up

step 2: partial sums move up

$$\text{tree: } 2 \log_2 p \cdot (\alpha + n / \beta)$$

$$\text{ring: } 2(p-1)\,\alpha + 2(p-1)/p \cdot n / \beta$$

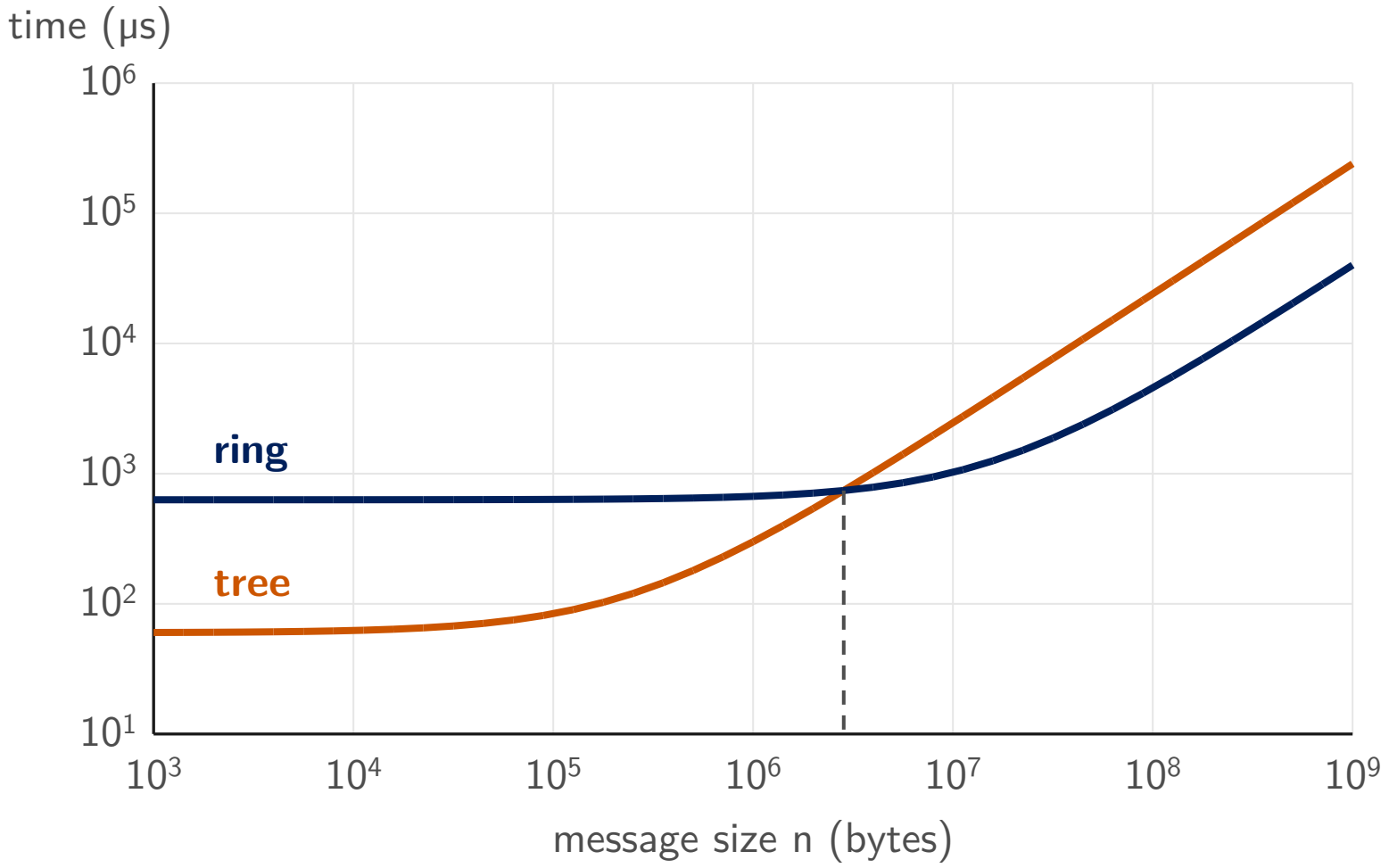
$$p = 64\text{: } 12 \text{ steps versus } 126$$

Key idea

Trees win on **latency** (log p steps); rings win on **bandwidth**.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Ring versus tree: which message sizes?



$p = 64$, $\alpha = 5$ μs

$\beta = 50$ GB/s

crossover $\approx$ 2.8 MB

small n: tree wins

large n: ring wins

IN PRACTICE

NCCL picks ring or tree **per call**; `NCCL_ALGO` overrides it.

Source: Cost models: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005. Values assumed.

All-reduce = reduce-scatter + all-gather

reduce-scatter
n bytes in $\rightarrow$ n/p summed slice
$(p-1)\alpha + (p-1)/p \cdot n/\beta$

$+$

all-gather
n/p slice in $\rightarrow$ n bytes out
$(p-1)\alpha + (p-1)/p \cdot n/\beta$

$$= \text{all-reduce: } 2(p-1)\alpha + 2(p-1)/p \cdot n \;/\; \beta$$

In our pipeline

Preview of Week 12 (ZeRO, FSDP): reduce-scatter the gradients, update **your slice** only, all-gather the parameters when needed.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005. Rajbhandari et al., "ZeRO," SC 2020.

Check yourself

- 4 GPUs, n $=$ 400 MB, $\beta = 450$ GB/s, $\alpha = 5$ µs. Ring all-reduce time?
- Going from 8 to 64 GPUs: what happens to each term of the ring cost?
- 64 GPUs all-reduce a 4 KB tensor. Ring or tree? Why?

Part 3

Bandwidth, asynchrony, ordering

How do we measure a collective — and why do programs hang?

Algorithm bandwidth and bus bandwidth

$$\text{algbw} = S \,/\, t$$

$$\text{busbw} = \text{algbw} \times \text{factor}$$

Collective	busbw factor
All-reduce	$2(p-1)/p$
Reduce-scatter	$(p-1)/p$
All-gather	$(p-1)/p$
Broadcast, reduce	1

Key idea

algbw: what the application sees. **busbw**: comparable with the **link speed**, whatever p is.

Source: NVIDIA nccl-tests, doc/PERFORMANCE.md (algorithm and bus bandwidth).

Why the bus-bandwidth factor?

$$\text{ideal ring:}\quad t = 2(p-1)/p \cdot S \;/\; \beta$$

$$\Rightarrow\quad \beta = (S \;/\; t) \cdot 2(p-1)/p = \text{busbw}$$

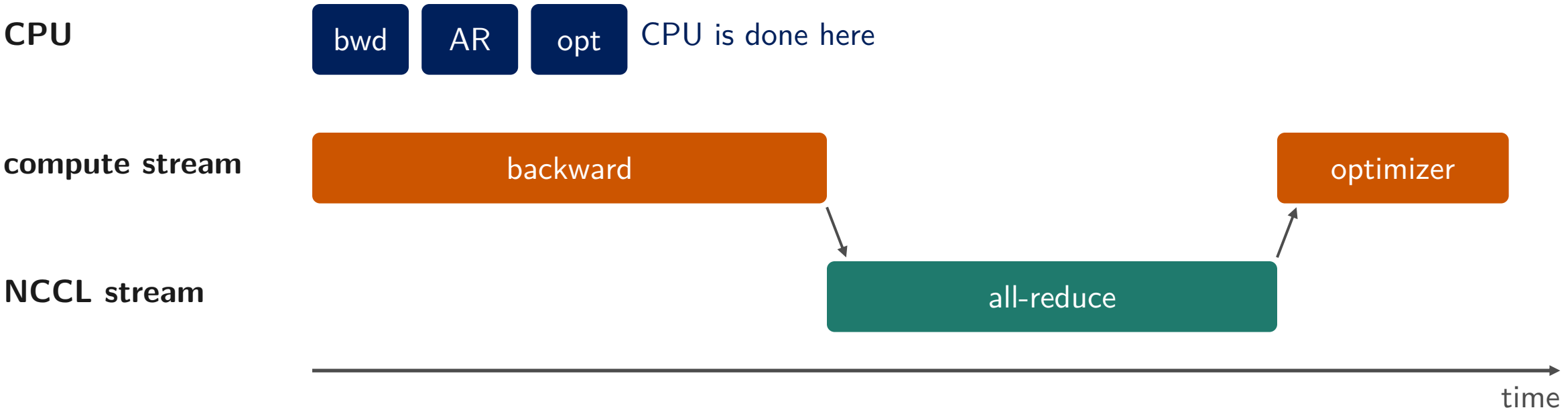
- Example (illustrative): p = 8, S = 1 GB, t = 4.0 ms
- algbw = 1 GB / 4.0 ms = **250 GB/s**
- busbw = 250 × 1.75 = **437.5 GB/s** ≈ 97% of NVLink's 450 GB/s per direction

Key idea

Compare **busbw** with the link's per-direction bandwidth: that ratio is your efficiency.

Source: NVIDIA nccl-tests, doc/PERFORMANCE.md (algorithm and bus bandwidth). NVLink: H100 datasheet.

NCCL calls are enqueued on CUDA streams



Watch out

When `all_reduce` returns, the sum is **queued**, not **done**. The streams keep the order.

Source: NCCL User Guide, "CUDA Stream Semantics"; PyTorch documentation, torch.distributed (v2.9).

async_op, wait(), and synchronize

```
work = dist.all_reduce(x, async_op=True)
y = model_part(z)          # independent work
work.wait()                # stream waits for x
x.mul_(1 / p)              # safe: ordered after
torch.cuda.synchronize() # CPU waits for GPU
print(x[0].item())
```

Overlap: start the collective, do **independent** work, then wait.

Call	Blocks the CPU?	Orders GPU work?
`all_reduce(x)`	no	yes (current stream)
`work.wait()` (NCCL)	no	yes (current stream)
`torch.cuda.synchronize()`	**yes**	everything done

Source: PyTorch documentation, torch.distributed (v2.9).

Ordering: every rank, same calls, same order

rank 0

```
all_reduce(a)
all_reduce(b)
```

rank 1

```
all_reduce(b)   # swapped!
all_reduce(a)
```



mismatched: rank 0's a is paired with rank 1's b

- Same size: a **silently wrong** sum (a + b).
- Different sizes: undefined — usually a **hang** until the timeout.

Watch out

Collectives are matched **by position**. Every rank must issue the same sequence.

Source: PyTorch documentation, torch.distributed (v2.9).

How ordering bugs sneak in

hangs

```
loss = compute_loss(batch)
if rank == 0:                    # BUG
    dist.all_reduce(loss)        # only rank 0
    print(loss.item() / p)
```

fixed

```
loss = compute_loss(batch)
dist.all_reduce(loss)            # every rank
if rank == 0:
    print(loss.item() / p)       # rank 0 only
```

- **Rank-dependent** `if` blocks
- Ranks with **different numbers** of batches
- Tensors whose **shape depends** on local data

RULE

Make collectives **unconditional**; branch only around printing and saving.

Timing a collective correctly

```
def time_ar(x, iters=20):
    for _ in range(5):
        dist.all_reduce(x)          # warm up
    torch.cuda.synchronize()
    dist.barrier()                  # align ranks
    t0 = time.perf_counter()
    for _ in range(iters):
        dist.all_reduce(x)
    torch.cuda.synchronize()        # wait for GPU
    dt = (time.perf_counter() - t0) / iters
    t = torch.tensor([dt], device="cuda")
    dist.all_reduce(t, op=dist.ReduceOp.MAX)
    return t.item()                 # slowest rank
```

- **Warm up**: first call sets up NCCL.
- **Synchronize** before reading the clock.
- **Many iterations**; divide.
- Report the **slowest** rank.

Watch out

Without `synchronize()` you time the **enqueue**, not the transfer.

Check yourself

- You time 20 all-reduces **without** `synchronize()`. What did you measure?
- $p = 8$, $S = 0.5$ GB, $t = 2.5$ ms. What are algbw and busbw?
- Rank 3 has 101 batches, the others 100. What happens at the end of the epoch?

IN-CLASS BUILD

Test NCCL results and completion

Work in pairs. This is the T2 checkpoint test.

Exercise: a correctness test

all-reduce test

```
p, r = dist.get_world_size(), dist.get_rank()
n = 1024
i = torch.arange(n, device="cuda",
                 dtype=torch.float32)
x = r + i                          # rank-dependent
dist.all_reduce(x)
want = p * (p - 1) / 2 + p * i  # closed form
assert torch.equal(x, want), r
```

- 1. Why is `x[i] = r + i` a good input? What is the sum for $p = 4$?
- 2. Write the check for `reduce_scatter_tensor` with the same input.
- 3. Write the check for `all_gather_into_tensor` when rank r sends `full((k,), r)`.

TIP

Use **integer-valued** FP32 so `torch.equal` is exact; test $p = 2$ and $p = 4$.

Exercise: solutions

Collective	Rank r's input	Expected output (p = 4)
`all_reduce`	$x[i] = r + i$	$6 + 4i \rightarrow [6, 10, 14, \ldots]$
`reduce_scatter_tensor`	$x[i] = r + i$	$6 + 4(r \cdot n/4 + j),\ j < n/4$
`all_gather_into_tensor`	`full((k,), r)`	$out[j] = j \,//\, k$

- Also test **p = 1**, n **not divisible** by p, and BF16 (needs a tolerance above 256).
- Random inputs: rings and trees sum in **different orders** → use `allclose`, not `equal`.

Check

A test that only checks "no crash" would pass a **swapped-order** bug. Check values.

Testing completion, then bandwidth

```
for e in range(4, 31, 2):          # 16 B..1 GB
    S = 2 ** e
    x = torch.zeros(S // 4, device="cuda")
    t = time_ar(x)
    alg = S / t / 1e9                    # GB/s
    bus = alg * 2 * (p - 1) / p
    if r == 0: print(S, t * 1e6, bus)
```

- Sizes **16 B** $\rightarrow$ **1 GB**, $\times 4$ apart
- Prints bytes, µs, busbw
- Small: time $\approx$ **constant**
- Large: busbw **plateaus**

In our pipeline

Same harness on every rank; only rank 0 prints. The table is your data for the α–β fit next lecture.

Compare with the NVLink figure

Message size	Regime	What busbw should look like
$\leq$ 64 KB	latency-bound	small; time $\approx$ constant
137 KB (our MLP)	latency-bound	far below the link
$\geq$ 256 MB	bandwidth-bound	plateau near 450 GB/s

$$\text{ceiling} = 900\ \text{GB/s} \div 2 = 450\ \text{GB/s per direction}$$

T2 CHECKPOINT (THIS WEEK)

Correct results for all three collectives at p = 2 and p = 4, a completion test, and a busbw sweep compared with 450 GB/s.

Source: H100 datasheet, NVLink 4 (900 GB/s bidirectional per GPU); NVIDIA nccl-tests, doc/PERFORMANCE.md (algorithm and bus bandwidth).

Key takeaways

- NCCL runs collectives as **GPU kernels**; one rank per GPU.
- **Ring all-reduce** costs $2(p-1)\alpha + 2(p-1)/p \cdot n/\beta$.
- All-reduce = **reduce-scatter + all-gather** (the halves ZeRO and FSDP use).
- The ring is **bandwidth-optimal**; trees need only **log p** steps.
- **busbw** = algbw $\times\ 2(p-1)/p$ is comparable with the link speed.
- Calls are **enqueued on streams**: synchronize before timing; test **values**.
- Every rank must call the **same collectives in the same order**.

Next time, and readings

When	What
This week	T2 checkpoint: correct collectives on small tensors
Lecture 2 (this week)	Multi-server communication: topology, launch, timeouts, α–β fit
After Lecture 2	Lab 4 due
Week 12	Data parallelism and sharding (DDP, ZeRO/FSDP)

READ BEFORE LECTURE 2

NCCL User Guide: *Collective Operations*, *Communicators*, *CUDA Stream Semantics*.
PyTorch *Distributed Overview*. Thakur et al., IJHPCA 2005 (Sections 1–4).